High-Level Synthesis for Multi-Core Systems

Md Mostafizur Rahman
Electronic Engineering
Hochschule Hamm-Lippstadt (HSHL)
Lippstadt, Germany
md-mostafizur.rahman@stud.hshl.de

***Abstract*—High-Level Synthesis (HLS) translates behavioral descriptions in C/C++, SystemC or similar languages into register-transfer level (RTL) hardware. Extending this flow to multi-core systems increases a number of decisions that are not present in a common single datapath design. The data-flow graph has to be partitioned over processing elements (PEs), operations and data transfers have to be scheduled and each operation has to be bound to a functional unit, taking into account communication and resource sharing. In this paper we present the mathematical foundations and formal model of multi-core HLS, and then describe a communication-aware list-scheduling algorithm as well as its runtime and correctness analysis. A C++ implementation demonstrates operation-to-cycle and operation-to-PE mapping for a 2×2 matrix multiplication graph and the greedy output motivates communication-aware partitioning. Two further experiments connect the model to practice: a POSIX-thread execution on an eight-core AMD Ryzen 7 machine measures non-ideal speedups of 1.70 and 2.31 for two and four workers and a Vivado synthesis comparison shows a 19.1% LUT reduction when mutually exclusive datapaths share hardware. The results suggest that parallelism is not enough and efficient multi-core HLS has to consider latency, communication and area together.**

***Index Terms*—high-level synthesis, multi-core systems, list scheduling, data-flow graph, resource binding, RTL, FPGA**

I. Introduction

Modern embedded applications in signal processing, image processing, automotive control, and machine learning require high throughput under tight area and energy constraints. A single processor or accelerator may not provide enough parallelism, and manually designing multiple RTL datapaths that work together is time consuming and error prone. High-Level Synthesis (HLS) raises the design abstraction level by mapping an algorithmic description into hardware structures and a controller [1], [2], [9]. The classical flow does compilation, data-width analysis, resource allocation, operation scheduling, resource binding and RTL generation.

A multi-core HLS target is composed of two or more processing elements (PEs) that can be identical or heterogeneous. Therefore, HLS must decide not only *when* an operation executes and *which* functional unit implements it but also *where* it executes. Values crossing PE boundaries require interconnect, communication scheduling and synchronization. These choices are coupled: a balanced partition may create many cross-PE edges, but aggressive resource sharing may result in area penalties, multiplexers, and controller complexity.

The paper satisfies the seminar requirements from mathematical foundations to executable implementation and RTL interpretation. Its contribution is 1) a formal communication and data flow model; 2) a communication-aware list scheduler with complexity and correctness analysis; 3) a C++ scheduling implementation and a POSIX multi-core experiment with measured speedup and efficiency; and 4) an FPGA resource-sharing experiment that quantifies the area effect of allocation and binding. The multi-mode approach of Casseau and Le Gal [5] is the main reference for joint scheduling, binding and path-cost awareness. Although this work targets timewise mutually exclusive modes instead of simultaneous active PEs, its treatment of operators, registers, multiplexers and controller cost is directly applicable to shared resource decisions in multi-core HLS.

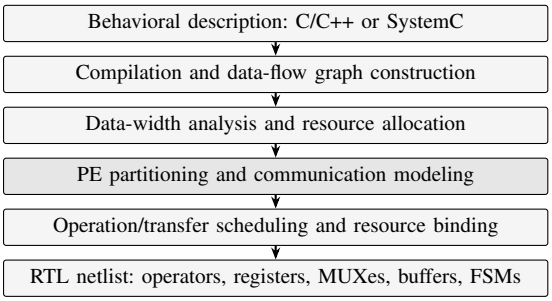

Fig. 1. HLS flow extended with PE partitioning and communication scheduling (shaded stage).

II. Mathematical Foundations and Formal Model

A. Data-Flow Graph and Scheduling Bounds

The computational kernel is represented by a directed acyclic data-flow graph (DFG)

$$G = (V, E), \qquad E \subseteq V \times V, \tag{1}$$

where each node $v_i \in V$ is an arithmetic or logical operation with latency $d_i \in \mathbb{Z}^+$ clock cycles. An edge $(v_i, v_j) \in E$ means that v_j consumes a value produced by v_i. The DFG exposes both dependencies and parallel operations, making it the central representation for scheduling and binding [3], [4].

The earliest feasible start time of an operation is computed by an as-soon-as-possible (ASAP) traversal:

$$t_{\mathrm{ASAP}}(v_i) = \begin{cases} 1, & \mathrm{pred}(v_i) = \emptyset, \\ \max\limits_{v_p \in \mathrm{pred}(v_i)} \big(t_{\mathrm{ASAP}}(v_p) + d_p\big), & \text{otherwise.} \end{cases} \tag{2}$$

For a latency bound λ, an as-late-as-possible (ALAP) backward traversal provides the latest start time that does not violate the bound. The mobility

$$\mu(v_i) = t_{\mathrm{ALAP}}(v_i) - t_{\mathrm{ASAP}}(v_i) \tag{3}$$

measures scheduling freedom; a node with zero mobility is critical and receives high scheduling priority. The parallel speedup across k PEs is

$$S(k) = T_1/T_k, \tag{4}$$

where T_1 is the single-core and T_k the k-core latency; ideally $S(k) \le k$, and communication overhead reduces it below this bound.

B. Multi-Core Decision Variables

Let $P = \{P_1, \ldots, P_m\}$ be the available PEs and let F_j be the functional units on P_j. Multi-core HLS determines four mappings:

$$\pi : V \to P \quad \text{(partition)}, \tag{5}$$

$$s : V \to \mathbb{Z}^+ \quad \text{(start control step)}, \tag{6}$$

$$\beta : V \to \bigcup\nolimits_j F_j \quad \text{(functional-unit binding)}, \tag{7}$$

$$\gamma : E_c \to \mathbb{Z}^+ \quad \text{(communication schedule)}, \tag{8}$$

where $E_c = \{(v_i, v_j) \in E \mid \pi(v_i) \neq \pi(v_j)\}$ is the set of cut edges. If δ_{ij} is the communication delay for a cut edge, each dependency must satisfy

$$s(v_j) \;\ge\; s(v_i) + d_i + \begin{cases} \delta_{ij}, & \pi(v_i) \neq \pi(v_j), \\ 0, & \pi(v_i) = \pi(v_j). \end{cases} \tag{9}$$

Resource constraints prevent more simultaneous operations of a type than the selected PE can execute: if $R_{j,t}$ is the number of type-t units on P_j, then for every control step k,

$$\sum_{v_i \in V_t : \pi(v_i) = P_j} x_{i,k} \;\le\; R_{j,t}, \tag{10}$$

where $x_{i,k} = 1$ when v_i occupies a type-t unit at step k. A practical objective is a weighted combination of makespan Λ, area A, dynamic energy E_{dyn}, and communication volume C:

$$\min\; J = w_\Lambda \Lambda + w_A A + w_E E_{\mathrm{dyn}} + w_C C. \tag{11}$$

Exact joint optimization is combinatorial; HLS tools therefore use heuristics, mathematical programming for smaller graphs, or hybrid design-space exploration [6], [10].

III. Communication-Aware List Scheduling and Binding

List scheduling is suitable for a seminar implementation because it is deterministic, understandable, and scalable [7]. At each control step it forms a set of ready nodes whose predecessors have completed, sorts them by priority, and binds them to free resources. The priority used in this work is

$$\mathrm{prio}(v_i) = \frac{1}{\mu(v_i) + 1} + \alpha_1 w_i + \alpha_2 |\mathrm{succ}(v_i)|, \tag{12}$$

where w_i is the operation word length. The first term favors critical nodes, the second favors expensive wide operations, and the third favors nodes whose results unlock more work. For each candidate PE, a communication-aware binding score is evaluated:

$$\chi(P_j, y, v_i) = \omega_{\mathrm{op}} + \omega_{\mathrm{path}} + \alpha_3 \delta_{\pi,i}. \tag{13}$$

Here $y \in F_j$ is a compatible free unit, ω_{op} is the operator overcost, and ω_{path} covers registers, multiplexers, and control. The last term sums communication penalties from already placed predecessors. This extends the path-aware joint scheduling and binding idea of [5]: the cheapest arithmetic operator is not necessarily the cheapest overall hardware path.

Algorithm 1: Communication-aware multi-core list scheduling

1) Compute ASAP, ALAP, and mobility for every node.
2) Set $k \leftarrow 1$ and mark all nodes unscheduled.
3) Build the ready set from nodes whose predecessors have finished (enforces (9)).
4) Sort ready nodes in descending order using (12).
5) For each node, inspect compatible free units on all PEs and select the minimum-cost choice from (13).
6) Assign (s, π, β), reserve the unit for d_i cycles, and add communication delay to cut edges.
7) Increment k and repeat until all nodes are scheduled.

Fig. 2. Formal description of the implemented scheduling algorithm.

A. Runtime and Overhead

Let $n = |V|$, $m = |P|$, and let at most R compatible unit slots be inspected per PE. Building and sorting a ready list costs $\mathcal{O}(n \log n)$ per control step; evaluating PE/resource choices costs $\mathcal{O}(nmR)$. With at most $\lambda \le n$ control steps in the worst case,

$$T(n) = \mathcal{O}\big(n^2 \log n + n^2 m R\big) = \mathcal{O}\big(n^2(\log n + mR)\big). \tag{14}$$

For a fixed architecture, m and R are constants, giving the familiar quadratic list-scheduling behavior. More accurate joint binding can use weighted bipartite matching, as in [5]; this improves interconnect quality but increases synthesis runtime (reported 10–94%, average 41%, of total runtime; synthesis times 0.2 s–167 s, a 2.6× increase over a spatially chained baseline at equivalent quality). Hardware overhead also matters: cross-PE values need buffers, shared units need larger multiplexers, and additional control states increase decoder logic; controller area is $\approx$16% of datapath area in multi-core designs vs. $\approx$10% in single-core HLS [5].

B. Conditions for Correctness

A generated schedule is valid if four in variants hold. First, each operation begins only after all predecessor results are available, including the delay of any cut edge. Second, no functional unit is allocated overlapping operations beyond its initiation capability. Third, the bound operation is compatible with the required arithmetic type and data width. Fourth, every scheduled inter-PE value has a storage or transport resource, and is not overwritten before it is read by its consumer.

The implemented ready-set test enforces the first invariant while per-cycle resource reservation resources reservations the second .enforce the second. By induction over control steps, all Operations selected at step k have completed predecessors and available resources, thus the final mapping is precedence- and resource-correct. This does does not prove optimally, but it distinguishes a valid heuristic schedule from an invalid fast schedule that ignores data arrival.

C. Alternative Scheduling Formulations

Force-directed scheduling (FDS) treats the feasible control steps of each operation as a probability distribution and flattens resource demand over time [6]. If v_i may execute in any step from $a_i = t_{\mathrm{ASAP}}(v_i)$ to $l_i = t_{\mathrm{ALAP}}(v_i)$, its probability is

$$p_{i,k} = \begin{cases} 1/(l_i - a_i + 1), & a_i \le k \le l_i, \\ 0, & \text{otherwise}, \end{cases} \tag{15}$$

and the distribution graph $DG_{t,k} = \sum_{v_i \in V_t} p_{i,k}$ estimates type-t demand at step k. A multi-core extension maintains one distribution per PE and adds a force for communication edges. Integer linear programming (ILP) encodes assignment, step, and binding as binary variables with linear constraints and objective (11); it is exact but grows exponentially, so it suits small or critical subgraphs. Table I summarizes the trade-off.

TABLE I
COMPARISON OF COMMON HLS SCHEDULING APPROACHES

Method	Solution quality	Typical cost	Suitable use
List	Heuristic	Low–medium	Large DFGs, latency
FDS	Heuristic	Medium–high	Balanced resources
ILP	Exact if solved	Very high	Small/critical DFGs

IV. C++ IMPLEMENTATION AND APPLICATION EXAMPLE

A. Core Scheduling Implementation

Listing 1 shows the core loop of the implemented C++ scheduler (`sched.cpp`). Each `Operation` stores predecessor identifiers, latency, and the assigned step and core. `predecessorsDone` enforces constraint (9) with δ_{ij}=0; the greedy loop assigns each ready operation to the first PE with a free functional unit (`numberOfCores` = 2, `functionalUnitsPerCore` = 2).

Listing 1. Core of the C++ list scheduler (`sched.cpp`).

```
bool predecessorsDone(const Operation& op,
    const vector<Operation>& ops, int currentStep) {
  for (int p : op.preds) {
    if (ops[p].step == -1) return false;
    if (ops[p].step + ops[p].latency > currentStep)
      return false;
  }
  return true;
}
// greedy scheduling loop
for (int step = 1; !allScheduled(ops); step++) {
  vector<int> usedFU(numberOfCores, 0);
  for (auto& op : ops) {
    if (op.step != -1) continue;
    if (!predecessorsDone(op, ops, step)) continue;
    for (int core = 0; core < numberOfCores; core++)
      if (usedFU[core] < functionalUnitsPerCore) {
        op.step = step; op.core = core;
        usedFU[core]++; break;
      }
  }
}
```

Compiled with `g++ -O2 sched.cpp -o sched`, the scheduler assigns the eight multiplications of the 2×2 matrix product evenly to PE1 and PE2 in steps 1–2, but assigns all four additions to PE1 in steps 3–4, since PE1 is just the first core with a free unit. Two of these additions (v_{11}, v_{12}) use products calculated on PE2. According to (9) with δ_{ij}=1 each such cut edge would delay the consumer and drop $S(2)$ below the ideal value. Thus, the greedy output shows experimentally *why* the binding score (13) needs its communication term.

B. 2×2 Matrix Multiplication DFG

For C=AB with 2x2 matrices, each of the four output elements need 2 multiplications (latency 2) and 1 addition (latency 1), with 12 DFG nodes. A row-based partition removes all transfers: PE0 computes C_{00} and C_{01}, and PE1 computes C_{10} and C_{11}. product-to-sum edges are then local, so $\delta_{ij} = 0$ for the entire graph.

Each PE contains two multiplier slots and one adder, so it takes two cycles to calculate a pair of products and one cycle to add them. A PE performing all four outputs takes T_1=12 cycles, while two row-partitioned PEs performing two outputs each take T_2=6 cycles. From (4) the ideal schedule yields

$$S(2) = \frac{T_1}{T_2} = \frac{12}{6} = 2.0. \tag{16}$$

Table II shows the cycle-level schedule. This small example illustrates why partitioning and scheduling cannot be independent: the same number of operations produces different latency once communication edges are introduced.

TABLE II
ROW-PARTITIONED TWO-PE SCHEDULE FOR THE 2×2 PRODUCT

Cycle(s)	PE0 (row 0)	PE1 (row 1)
1–2	two products for C_{00}	two products for C_{10}
3	add C_{00}	add C_{10}
4–5	two products for C_{01}	two products for C_{11}
6	add C_{01}	add C_{11}

C. POSIX Multi-Core Model

Our second implementation (`matrix_pthread.c`) employs POSIX threads as a functional model for PEs: each `pthread` corresponds to one PE, row blocks of a 64×64 matrix multiplication reflect the DFG partition, `usleep` models hardware operator latency, and `pthread_join` models synchronization, the software analogue of δ_{ij} in (9). The program was compiled with `gcc -O2 -pthread` and executed on an eight-core AMD Ryzen 7 7435HS running Linux. Six runs were performed and the median wall-clock time was recorded using `CLOCK_MONOTONIC`.

Table III shows that two workers achieved a speedup of 1.70 and four workers 2.31, both below ideal. The efficiency

TABLE III
POSIX MATRIX-MULTIPLICATION RESULTS (N=64, MEDIAN OF SIX RUNS)

Workers K	Time (s)	Speedup $S(K)$	Efficiency $\eta(K)$
1	0.002286	1.00	100.0%
2	0.001346	1.70	85.0%
4	0.000992	2.31	57.8%

$\eta(K) = S(K)/K$ falls from 85.0% at two workers to 57.8% at four. The deviation can be expressed as an excess time

$$\Delta T_K = T_K - \frac{T_1}{K}. \tag{17}$$

For two workers $\Delta T_2 = 0.000203\,\text{s}$; for four workers $\Delta T_4 = 0.000421\,\text{s}$. The larger value at K=4 indicates that management and synchronization grow relative to the smaller amount of useful work per worker. Across the six runs, $S(2)$ ranged from 1.48 to 1.75 and $S(4)$ from 2.05 to 2.75 due to operating-system scheduling variability. These values should not be read as pure interconnect delay—the model also includes OS scheduling and timer granularity—but they provide a quantitative reason for including the communication/overhead terms in objective (11).

V. FPGA RESOURCE SHARING AND RTL OUTPUT

A. Vivado Resource-Sharing Experiment

The third experiment evaluates the area side of HLS allocation and binding. Two mutually exclusive arithmetic modes were described in VHDL with the same interface (8-bit inputs a, b, c, d, one mode signal, 17-bit output y; 50 I/O pins):

$$\text{Mode 0:} \quad y = (a \times b) + (c \times d), \tag{18}$$

$$\text{Mode 1:} \quad y = (a + b) \times (c + d). \tag{19}$$

A *separate* implementation retains dedicated arithmetic for both modes (two multipliers, three adders, output selected by mode). A *shared* implementation reuses one multiplier across modes through multiplexed inputs:

```
op1 <= resize(unsigned(a),9) when mode='0'
       else resize(unsigned(a),9)
          + resize(unsigned(b),9);
op2 <= resize(unsigned(b),9) when mode='0'
       else resize(unsigned(c),9)
          + resize(unsigned(d),9);
shared_mult <= op1 * op2;
```

Vivado synthesis reported 262 LUTs for the separate version and 212 LUTs for the shared version with identical I/O (Table IV), a reduction of

$$\frac{262 - 212}{262} \times 100\% = 19.1\%. \tag{20}$$

This is not a claim that every multi-core architecture should share one functional unit between simultaneously active PEs. Instead, it demonstrates the binding principle of [5]: when lifetimes or operating modes do not overlap, reuse reduces area, while multiplexers and control determine the final benefit. The 19.1% saving is consistent with the 11–58% ASIC area savings reported in [5]; the more modest FPGA gain reflects the coarser granularity of LUT-based logic [9].

TABLE IV
VIVADO SYNTHESIS COMPARISON FOR SEPARATE AND SHARED DATAPATHS

Architecture	LUTs	I/O pins	LUT change
Separate datapaths	262	50	–
Shared resources	212	50	-19.1%

B. Schedule to RTL Netlist

After scheduling and binding, HLS emits a structural RTL architecture. Each PE consists of functional units, input and pipeline registers, a local register file, multiplexers for operand selection, and an FSM that asserts register-enable and multiplex-select signals [2]. A cut edge in the DFG is mapped to a communication element such asa register, FIFO, bus transaction or network-on-chip packet. schedule defines the state in which each transfer or operation takes place; the binding defines physical connectivity. A representative entity skeleton for a shared adder:

```
entity add_16 is
  port(
    a, b : in  std_logic_vector(15 downto 0);
    clk  : in  std_logic;
    en   : in  std_logic;
    sum  : out std_logic_vector(15 downto 0));
end add_16;
```

The FSM drives `en` high only in the control step to which the addition is bound—the same multiplexed-input principle as the `shared_mult` path above. case of the matrix example, the row-based partition leads to two PE entities and no cross-PE data buffers. Each PE contains two multipliers, one adder, local registers, a multiplexer tree and one FSM controller. The main reference shows why register and multiplexer paths need to be considered at binding time: operator sharing can be overcome by wide or high-input-count multiplexers [5]. Data-width analysis further decreases both operator area and interconnect width [8].

VI. DISCUSSIONS AND LIMITATIONS

The three experiments show complementary aspects of multi-core HLS. The logical mapping DFG nodes to cycles, PEs and resources is exposed in the C++ scheduler and its greedy output exposes what goes wrong without communication awareness. that more workers increase runtime, but do not guarantee optimal speedup. The FPGA experiment suggests that area is reduced with sharing, but may increase multiplexing and control. They together form the basis of the main design principle: that parallelism, communication and resource sharing should be optimized at the same time.

A. Memory, interconnect and HLS directives

The DFG includes arithmetic dependencies, but memory can be the real bottleneck of a multi-core accelerator. If multiple PEs read the same single port array in one cycle, the schedule is infeasible even with free arithmetic units. HLS must model memory ports as resources, or transform storage via banking, replication, and local scratchpads. Row

partitioning gives each PE a disjoint output block, but both still read matrix B. The implementation can duplicate read-only data or arbitrate a shared memory, trading area for bandwidth. The interconnect has a similar effect: a register link is cheap for nearby PEs, a bus requires arbitration, and a network-on-chip adds routing and buffering, so δ_{ij} in (9) should depend on topology, transfer width, and contention. HLS directives reveal the same tradeoffs: pipelining increases throughput, unrolling exposes PE parallelism, array partitioning increases memory ports, and allocation limits encourage sharing [9].

B. Limitation

Some limitations exist. First, our scheduler is greedy and may miss a globally better partition or schedule. ILP produces exact solutions for small graphs but scales exponentially. Second, the POSIX experiment approximates PEs at the software-thread level, and does not model cycle-accurate interconnect arbitration or memory ports. Third, the Vivado sharing example uses mutually exclusive modes, thereby validating the binding and area argument rather than concurrent multi-PE operation. Finally, realistic many-core designs require memory-bank allocation, DMA scheduling and network-on-chip latency models; heterogeneous systems further require PE-specific latency and energy models. Modern frameworks such as ScaleHLS [10] show how multi-level compiler representations (MLIR) and automated design-space exploration address some of these scaling problems, reporting up to $768\times$ improvement over baseline Vivado HLS on computation kernels.

VII. Conclusions

This paper presented High-Level Synthesis for multi-core systems from mathematical modeling to implementation and RTL realization. A DFG formalizes operations and dependencies; the multi-core solution is defined by partition, schedule, binding and communication mappings. The communication-aware list scheduler has complexity $\mathcal{O}(n^2(\log n + mR))$ and was implemented in C++ for the 12-node matrix-multiplication graph. Its greedy output (all additions on one PE with two cross-PE input pairs) concretely showed why the binding score needs a communication term. The row-based partitioning removed all cut edges and achieved the best two-PE speedup $S(2){=}12/6{=}2.0$. The measured median speedups for two and four workers in the POSIX model on an eight-core AMD Ryzen 7 were 1.70 and 2.31, respectively, indicating non-ideal scalingand increasing synchronization overhead, with efficiency decreasing from 85.0% to 57.8%. A separate Vivado experiment resulted in a resource sharing that reduced LUT usage from 262 to 212 (19.1%). The final RTL contains functional units, registers, multiplexers, communication buffers and FSM controllers. The main takeaway is that HLS for multi-core systems is not only C-to-RTL translation but joint optimization of computation placement, timing, data movement and hardware cost.

References

[1] J. P. Elliott, *Understanding Behavioral Synthesis: A Practical Guide to High-Level Design*. Norwell, MA, USA: Kluwer Academic Publishers, 2000.

[2] P. Coussy and A. Morawiec, Eds., *High-Level Synthesis: From Algorithm to Digital Circuit*. Dordrecht, The Netherlands: Springer, 2008.

[3] S. Gupta, R. Gupta, N. Dutt, and A. Nicolau, *SPARK: A Parallelizing Approach to the High-Level Synthesis of Digital Circuits*. Boston, MA, USA: Springer, 2004.

[4] G. De Micheli, *Synthesis and Optimization of Digital Circuits*. New York, NY, USA: McGraw-Hill, 1994.

[5] E. Casseau and B. Le Gal, "Design of multi-mode application-specific cores based on high-level synthesis," *Integration, the VLSI Journal*, vol. 45, no. 1, pp. 9–21, Jan. 2012.

[6] P. G. Paulin and J. P. Knight, "Force-directed scheduling for the behavioral synthesis of ASICs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 8, no. 6, pp. 661–679, Jun. 1989.

[7] A. M. Sllame and V. Drabek, "An efficient list-based scheduling algorithm for high-level synthesis," in *Proc. Euromicro Symp. Digital System Design*, 2002, pp. 316–323.

[8] M. Stephenson, J. Babb, and S. Amarasinghe, "Bitwidth analysis with application to silicon compilation," in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation*, 2000, pp. 108–120.

[9] R. Nane *et al.*, "A survey and evaluation of FPGA high-level synthesis tools," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, no. 10, pp. 1591–1604, Oct. 2016.

[10] H. Ye, C. Hao, J. Cheng, H. Jeong, J. Huang, S. Neuendorffer, and D. Chen, "ScaleHLS: A new scalable high-level synthesis framework on multi-level intermediate representation," in *Proc. IEEE Int. Symp. High-Performance Computer Architecture (HPCA)*, 2022, pp. 741–755.